Exercise 4: Joins, UNION, Filtering, & Aggregates

Z 15 (81/142289)

1. SQL Joins

1. INNER JOIN

```
SELECT EmployeeID, FirstName, LastName, Department, Salary,
       ProjectID, ProjectName, Budget, Status
FROM Employees AS A
INNER JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	101	AI Development	100000	Completed
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
1	John	Doe	IT	70000	103	Cybersecurity Audit	75000	Pending
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Completed
6	Michael	Clark	Finance	80000	106	Risk Management	80000	Pending

2. LEFT JOIN

```
SELECT EmployeeID, FirstName, LastName, Department, Salary,
       ProjectID, ProjectName, Budget, Status
FROM Employees AS A
LEFT JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	103	Cybersecurity Audit	75000	Pending
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
4	David	Brown	IT	72000	NULL	NULL	NULL	NULL
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Completed
6	Michael	Clark	Finance	80000	106	Risk Management	80000	Pending
1	John	Doe	IT	70000	101	AI Development	100000	Completed

G.P.-S. 06/02

3. RIGHT Join

```
SELECT ProjectID, ProjectName, Budget, Status, EmployeeID, FirstName,
       LastName, Department, Salary
FROM Employees AS A
RIGHT JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID;
```

ProjectID	ProjectName	Budget	Status	EmployeeID	FirstName	LastName	Department	Salary
101	AI Development	100000	Completed	1	John	Doe	IT	70000
102	Employee Training	50000	Ongoing	2	Alice	Smith	HR	60000
103	Cybersecurity Audit	75000	Pending	1	John	Doe	IT	70000
104	Financial Analysis	90000	Ongoing	3	Bob	Johnson	Finance	75000
105	Market Expansion	65000	Completed	5	Emma	Wilson	Sales	65000
106	Risk Management	80000	Pending	6	Michael	Clark	Finance	80000

4. FULL OUTER JOIN

```
SELECT EmployeeID, FirstName, LastName, Department, Salary, ProjectID,
       ProjectName, Budget, Status
FROM Employees AS A
FULL OUTER JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID;
```

EmployeeID	FirstName	LastName	Department	Salary	ProjectID	ProjectName	Budget	Status
1	John	Doe	IT	70000	101	AI Development	100000	Completed
1	John	Doe	IT	70000	103	Cybersecurity Audit	75000	Pending
2	Alice	Smith	HR	60000	102	Employee Training	50000	Ongoing
3	Bob	Johnson	Finance	75000	104	Financial Analysis	90000	Ongoing
4	David	Brown	IT	72000	NULL	NULL	NULL	NULL
5	Emma	Wilson	Sales	65000	105	Market Expansion	65000	Completed
6	Michael	Clark	Finance	80000	106	Risk Management	80000	Pending

2. UNION & UNION ALL

5.

```
SELECT City AS Location
FROM Employees
UNION
SELECT Status
FROM Projects;
```

Location
New York
Los Angeles
Toronto
London
Sydney
Completed
Ongoing
Pending

6.

```
SELECT City AS Location
FROM Employees
UNION ALL
SELECT Status
FROM Projects;
```

Location
New York
Los Angeles
Toronto
London
Sydney
New York
Completed
Ongoing
Pending
Ongoing
Completed
Pending

3. Filtering Statements

7.

```
SELECT EmployeeID, FirstName, LastName, Department, Salary
FROM Employees
WHERE Salary >= 70000;
```

EmployeeID	FirstName	LastName	Department	Salary
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000
6	Michael	Clark	Finance	80000

8.

```
SELECT EmployeeID, FirstName, LastName, Department, Salary
FROM Employees
WHERE Department = 'IT' OR Department = 'Finance';
```

EmployeeID	FirstName	LastName	Department	Salary
1	John	Doe	IT	70000
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000
6	Michael	Clark	Finance	80000

9.

```
SELECT ProjectID, ProjectName, Budget, Status
FROM Projects
WHERE NOT Status = 'Completed';
```

ProjectID	ProjectName	Budget	Status
102	Employee Training	50000	Ongoing
103	Cybersecurity Audit	75000	Pending
104	Financial Analysis	90000	Ongoing
106	Risk Management	80000	Pending

10.

```
SELECT ProjectID, ProjectName, Budget, Status
FROM Projects
WHERE NOT Status = 'Completed' AND Budget > 70000;
```

ProjectID	ProjectName	Budget	Status
103	Cybersecurity Audit	75000	Pending
104	Financial Analysis	90000	Ongoing
106	Risk Management	80000	Pending

11.

```
SELECT EmployeeID, FirstName, LastName, Department, Salary, City
FROM Employees
WHERE City = 'New York' OR City = 'Toronto'
ORDER BY Salary DESC;
```

EmployeeID	FirstName	LastName	Department	Salary	City
6	Michael	Clark	Finance	80000	New York
3	Bob	Johnson	Finance	75000	Toronto
1	John	Doe	IT	70000	New York

12.

```
SELECT EmployeeID, FirstName, LastName, Department, Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 3;
```

Employee ID	FirstName	LastName	Department	Salary
6	Michael	Clark	Finance	80.000
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000

4. Aggregate Functions with GROUP BY & HAVING

13.

```
SELECT Department,
       SUM(Salary) AS TotalSalary
FROM Employees
Group by Department
ORDER BY TotalSalary DESC;
```

Department	TotalSalary
Finance	155000
IT	142000
Sales	65000
HR	60000

14.

```
SELECT City,
       AVG(Salary) AS AverageSalary
FROM Employees
GROUP BY City
HAVING AverageSalary > 65000;
```

City	AverageSalary
New York	75000
Toronto	75000
London	72000

15.

```
SELECT Department,
       COUNT(EmployeeID) AS EmployeeCount
FROM Employees
GROUP BY Department
HAVING EmployeeCount > 1;
```

Department	EmployeeCount
IT	2
Finance	2

16.
```
SELECT Status,
       COUNT(ProjectID) AS ProjectCount
FROM Projects
GROUP BY Status
HAVING ProjectCount >= 2;
```

Status	ProjectCount
Completed	2
Ongoing	2
Pending	2

17.
```
SELECT EmployeeID, FirstName, LastName,
       SUM(Budget) AS TotalProjectBudget
FROM Employees AS A
INNER JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID
GROUP BY EmployeeID,
         FirstName,
         LastName
HAVING TotalProjectBudget > 150000;
```

EmployeeID	FirstName	LastName	TotalProjectBudget
1	John	Doe	175000